—

Questão 1. Observe o código abaixo:

```c
#include <stdio.h>

int main() {
    int numero = 10;
    int *p = &numero;

    printf("numero = %d\n", numero);
    printf("p = %p\n", (void*)p);
    printf("*p = %d\n", *p);

    return 0;
}
```

a) O que está armazenado na variável `numero`?

10

b) O que está armazenado na variável `p`?

A variavel p e um ponteiro q esta indicando o endereço da variavel numero

c) O que significa `*p`?

Significa que a variavel criada e do tipo ponteiro que guarda o endereço de memoria ao inves de um dado

d) O que será apresentado pelo `printf("*p = %d\n", *p)`?

mostra o endereço de um espaço na memoria que o ponteiro encontrou para guardar o valor da variavel

e) Explique detalhadamente o que acontece se substituirmos:

*p = 50;

o ponteiro vai subistituir o valor que esta dentro da variavel por 50

Questão 2. Qual é a diferença entre os seguintes tipos?

Crie um programa que:

1. Declare uma variável `int numero = 100`.
2. Crie um ponteiro para essa variável.
3. Mostre o valor de `numero`.
4. Altere o valor de `numero` para `500` utilizando **somente o ponteiro**.
5. Mostre novamente o valor de `numero`.

Resultado esperado:
Antes: 100
Depois: 500

```
#include <stdio.h>
#include<stdlib.h>
int main()
{

int numero = 100;
int*n1 = &numero;

printf("valor do numero:%d\n",numero);

*n1 = 500;
printf("valor do numero:%d",numero);


return 0;
}
```

Questão 3. O programa utiliza dois ponteiros, um para cada variável (`a` e `b`). Primeiro, são exibidos os valores e os endereços de memória das variáveis. Em seguida, os valores são alterados por meio dos ponteiros, passando `a` para 100 e `b` para 200.

Questão 4. Observe o código abaixo e responda as questões:

```c
int *numero;

numero = malloc(sizeof(int));

free(numero);
```

a) O que é `numero`?

O ponteiro

b) O que `malloc(sizeof(int))` está fazendo, explique detalhadamente esse código?

O numero e um ponteiro que vai indicar um espaço de armazenamento na memoria nesse caso o espaco necessario n foi estipulado ao inves disso vamos definir uma alocacao dinamica o malloc serve para solicitar um espaco na memoria no valor de byte de um int

c) Em qual área da memória a variável **`numero`** fica armazenada?

pilha

d) Onde fica o espaço de memória reservado pelo `malloc`?

heap

e) Para que serve o `free()`?

Para liberar a memoria que foi reservada pelo malloc

Questão 5. Crie um programa que reserve dinamicamente espaço para **um inteiro**.

Depois:

1. Armazene o valor `100`.
2. Mostre o valor armazenado.
3. Mostre o endereço da memória alocada.
4. Libere a memória utilizando `free()`.

```c
#include <stdio.h>
```

```c
#include <stdlib.h>

int main() {
    int *p = malloc(sizeof(int));
    *p = 100;

    printf("O valor da variavel %d", *p);
    printf("\n O endereço é %p", p);
    free(p);

    return 0;

}
```

Questão 06 – Crie um programa em C que utilize um ponteiro para armazenar **10 números inteiros**.

Utilize a declaração:

```
int *numeros;
```

O programa deverá:

1. **Alocar memória** suficiente para armazenar 10 números inteiros utilizando `malloc`.
2. **Ler** 10 números digitados pelo usuário.
3. **Armazenar** os números na memória alocada.
4. **Exibir** todos os números armazenados.
5. **Liberar** a memória utilizada com `free()`.

```
#include <stdio.h>
#include <stdlib.h>

int main() {
   int *numeros;
   int i;


   numeros = (int *)malloc(10 * sizeof(int));
```

```
    if (numeros == NULL) {
        printf("Erro ao alocar memoria.\n");
        return 1;
    }


    printf("Digite 10 numeros inteiros:\n");

    for (i = 0; i < 10; i++) {
        printf("Numero %d: ", i + 1);
        scanf("%d", &numeros[i]);
    }


    printf("\nNumeros armazenados:\n");

    for (i = 0; i < 10; i++) {
        printf("%d ", numeros[i]);
    }


    free(numeros);

    return 0;
}
```

Questão 07 – Explique o que significa:

```
malloc(10 * sizeof(int));
```

Depois responda:

a) Quantos `int` estão sendo reservados?

10

b) Por que usamos `sizeof(int)`?

Porque o tamanho de um `int` **pode variar dependendo do sistema/compilador**.

c) A quantidade de bytes reservados é necessariamente 10?

Os `10` representam a quantidade de **elementos (`int`)**, e não a quantidade de bytes.

d) Se `sizeof(int)` for 4 bytes, quantos bytes serão reservados

40 bytes

Questão 08 – Crie um programa em C que declare uma variável inteira com o valor **10** e, em seguida, crie um ponteiro que aponte para essa variável.

O programa deverá mostrar o valor da variável **antes da alteração** e, utilizando o ponteiro, alterar seu valor para **50**. Ao final, exiba novamente o valor da variável, mostrando o resultado após a alteração.

```
#include <stdio.h>


int main() {
    int numero = 10;
    int *ponteiro;



    ponteiro = &numero;



    printf("Valor antes da alteracao: %d\n", numero);



    *ponteiro = 50;
```

```
    printf("Valor depois da alteracao: %d\n", numero);


    return 0;

}
```

Questão 09 – Faça um programa que leia dois números inteiros.

Crie dois ponteiros, um para cada número.

Utilizando os ponteiros, mostre:

- o valor dos dois números;
- o endereço de memória de cada número;
- a soma dos dois números por meio do ponteiro.

```
#include <stdio.h>

int main() {
    int numero1, numero2;
    int *ponteiro1, *ponteiro2;


    printf("Digite o primeiro numero: ");
    scanf("%d", &numero1);

    printf("Digite o segundo numero: ");
    scanf("%d", &numero2);


    ponteiro1 = &numero1;
    ponteiro2 = &numero2;


    printf("\nValor do primeiro numero: %d\n", *ponteiro1);
    printf("Valor do segundo numero: %d\n", *ponteiro2);


    printf("Endereco do primeiro numero: %p\n", (void *)ponteiro1);
    printf("Endereco do segundo numero: %p\n", (void *)ponteiro2);
```

```
    printf("Soma dos dois numeros: %d\n", *ponteiro1 + *ponteiro2);

    return 0;
}
```

Questão 10 – Crie um programa em C que leia **dois números inteiros** informados pelo usuário.

Em seguida, crie uma função chamada `trocar`, que receba como parâmetros os **endereços de memória dos dois números** e utilize ponteiros para trocar os valores entre eles.

Ao final, mostre os valores das duas variáveis **antes e depois da troca**, permitindo verificar o resultado da operação.#include <stdio.h>

```
void trocar(int *a, int *b) {
    int temp;

    temp = *a;
    *a = *b;
    *b = temp;
}

int main() {
    int numero1, numero2;

    printf("Digite o primeiro numero: ");
    scanf("%d", &numero1);

    printf("Digite o segundo numero: ");
```

```
    scanf("%d", &numero2);



    printf("\nAntes da troca:\n");
    printf("Numero 1: %d\n", numero1);
    printf("Numero 2: %d\n", numero2);



    trocar(&numero1, &numero2);



    printf("\nDepois da troca:\n");
    printf("Numero 1: %d\n", numero1);
    printf("Numero 2: %d\n", numero2);

    return 0;
}
```

DESAFIADORA – Crie um programa em C que utilize **alocação dinâmica de memória** com `malloc()` e `realloc()`.

O programa deverá:

1. Criar um ponteiro `int *numeros`.
2. Utilizar `malloc()` para reservar espaço para **5 números inteiros**.
3. Verificar se a memória foi alocada corretamente.
4. Armazenar os valores de **1 a 5** na memória alocada.
5. Utilizar `realloc()` para aumentar o espaço e permitir o armazenamento de **10 números inteiros**.
6. Preencher as novas posições com os valores de **6 a 10**.

7. Exibir os **10 números** na tela.
8. Ao final, utilizar `free()` para liberar a memória.